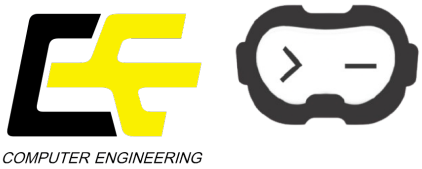


Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Akhir
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

Pengantar Mikrokontroler

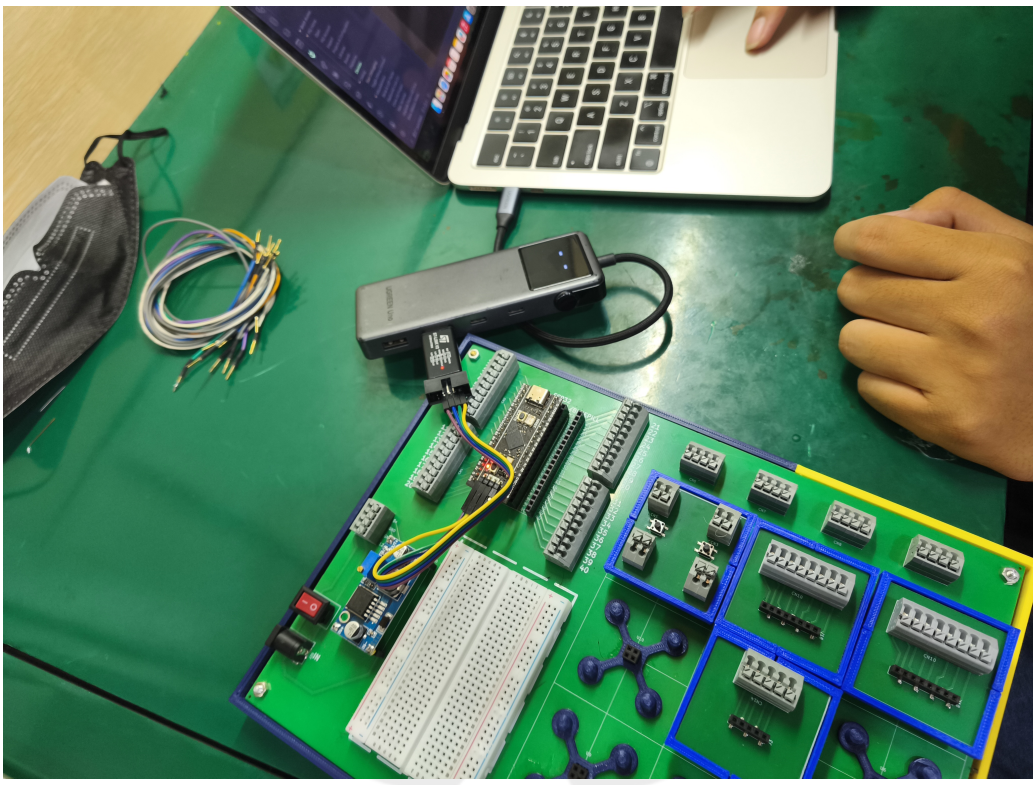
Naufal Ahimsa Raushanfekr - 5024241018

13 November 2026

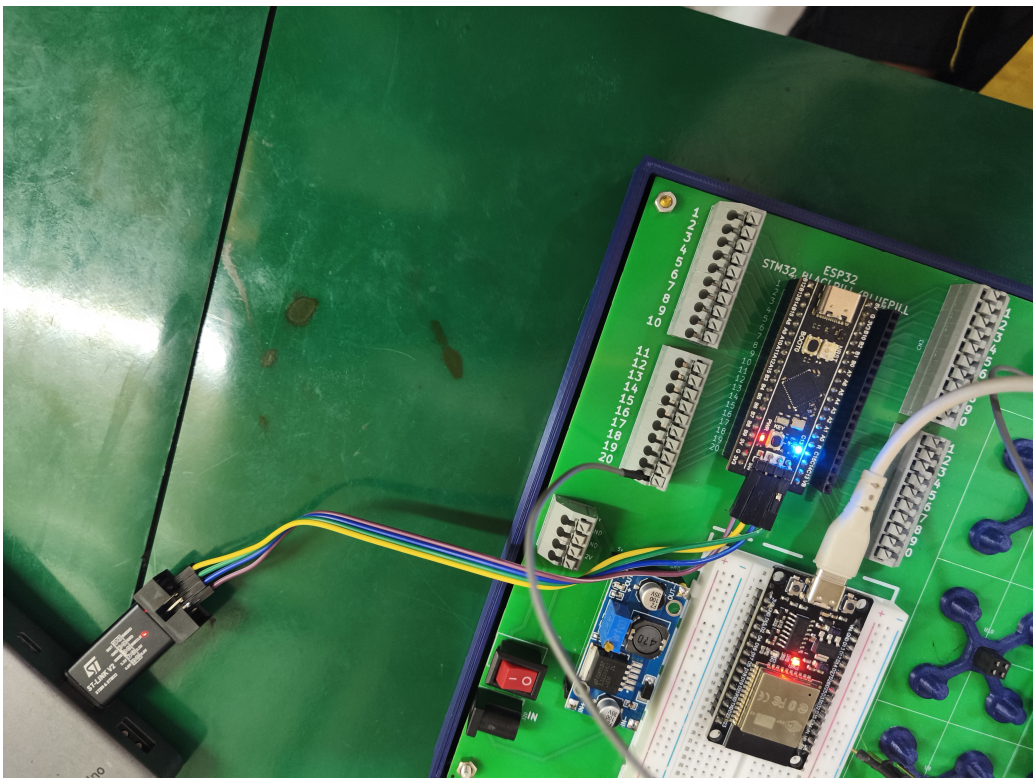
1 Langkah-Langkah Percobaan

1.1 PERCOBAAN 1: Pengenalan STM32 Blackpill (Blink)

1. Pastikan project Modul 1 sudah dibuat (Bagian D) dan isi platformio.ini sama seperti di atas
2. Hubungkan ST-Link ke Blackpill dengan kabel, lalu colokkan ST-Link ke komputer lewat USB. Cocokkan nama pinnya, karena letak pin ST-Link bisa berbeda-beda:

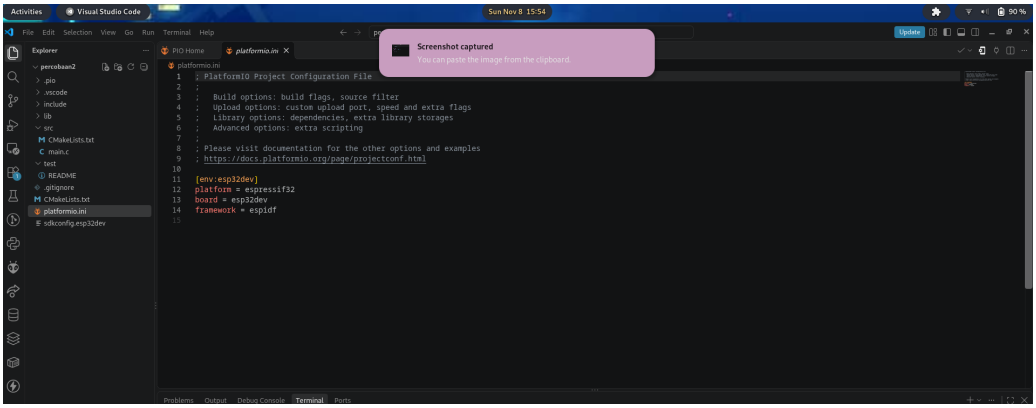


3. Tulis kode Blink di bawah pada src/main.cpp, lalu klik Build dan Upload
4. Lihat LED di board (PC13): nyala setengah detik, mati setengah detik, berulang terus

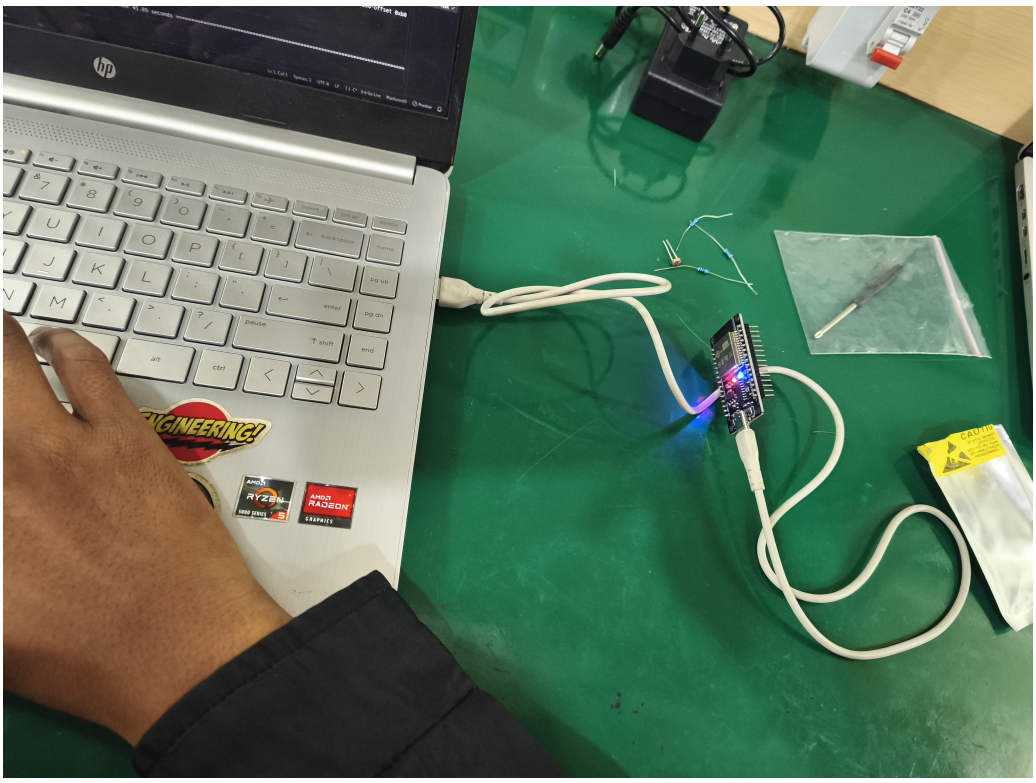


1.2 PERCOBAAN 2: Pengenalan ESP32 dengan Framework ESP-IDF (Blink)

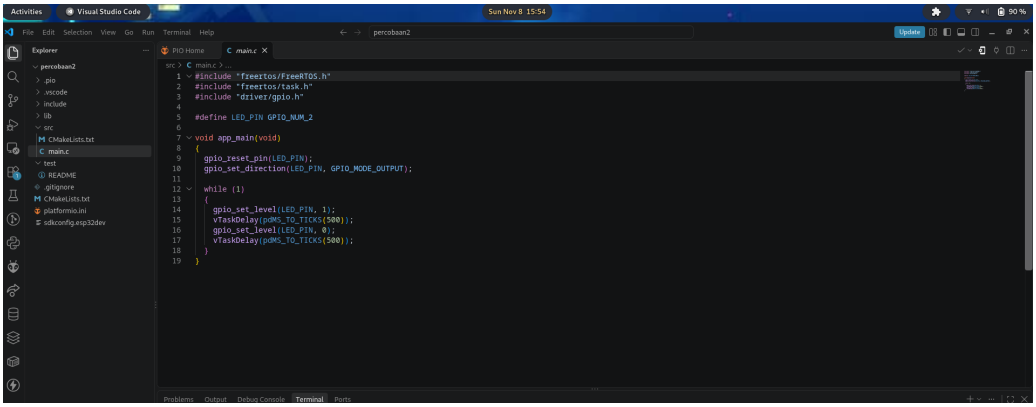
1. Ganti isi platformio.ini dengan konfigurasi di atas (board ESP32 DevKit, framework ESP-IDF)



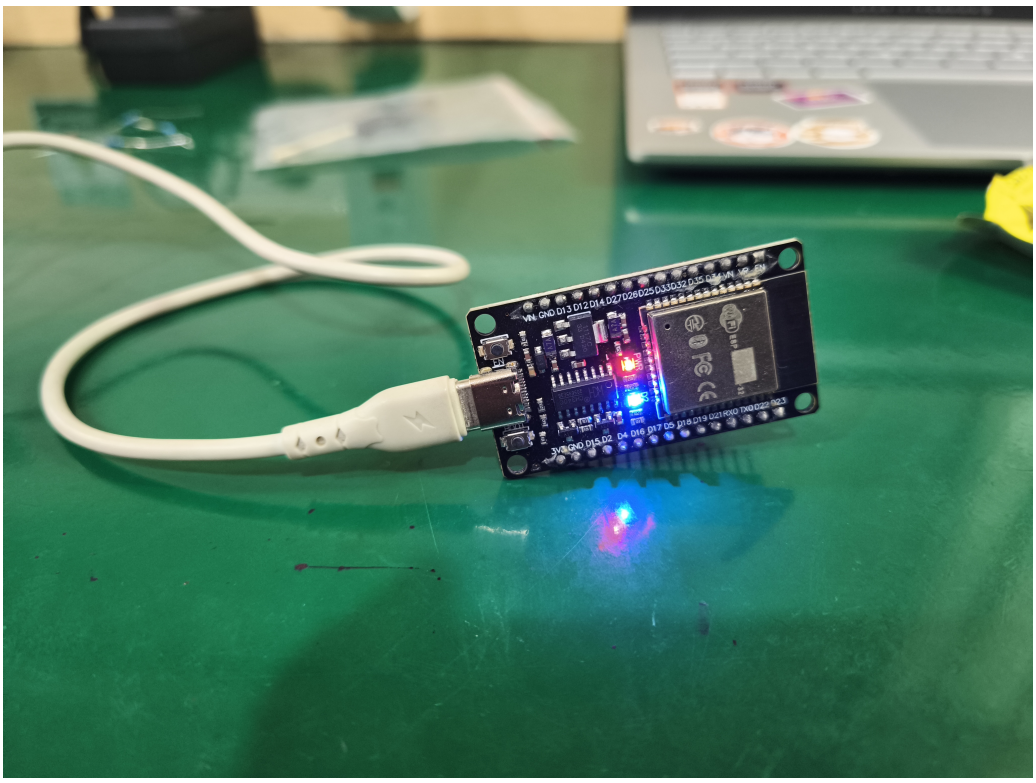
2. Colokkan ESP32 ke komputer lewat USB, lalu pastikan komputer mengenali port-nya



3. Hapus file src/main.cpp, buat file baru src/main.c, tulis kode Blink di bawah, lalu klik Build dan Upload

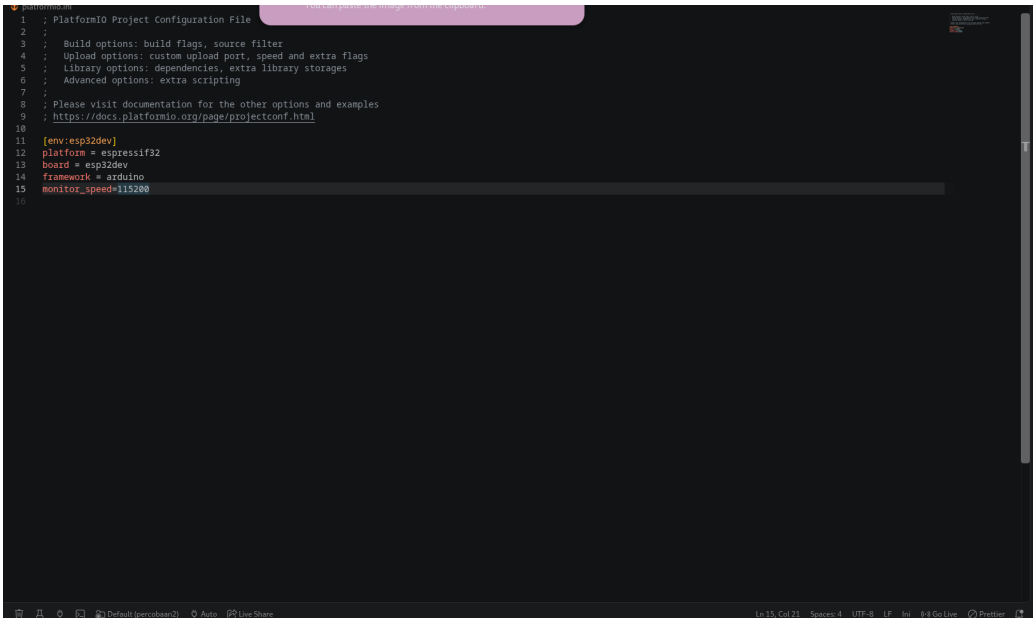


4. Analisis apa perbedaan memprogram dengan ESP-IDF dan dengan Arduino

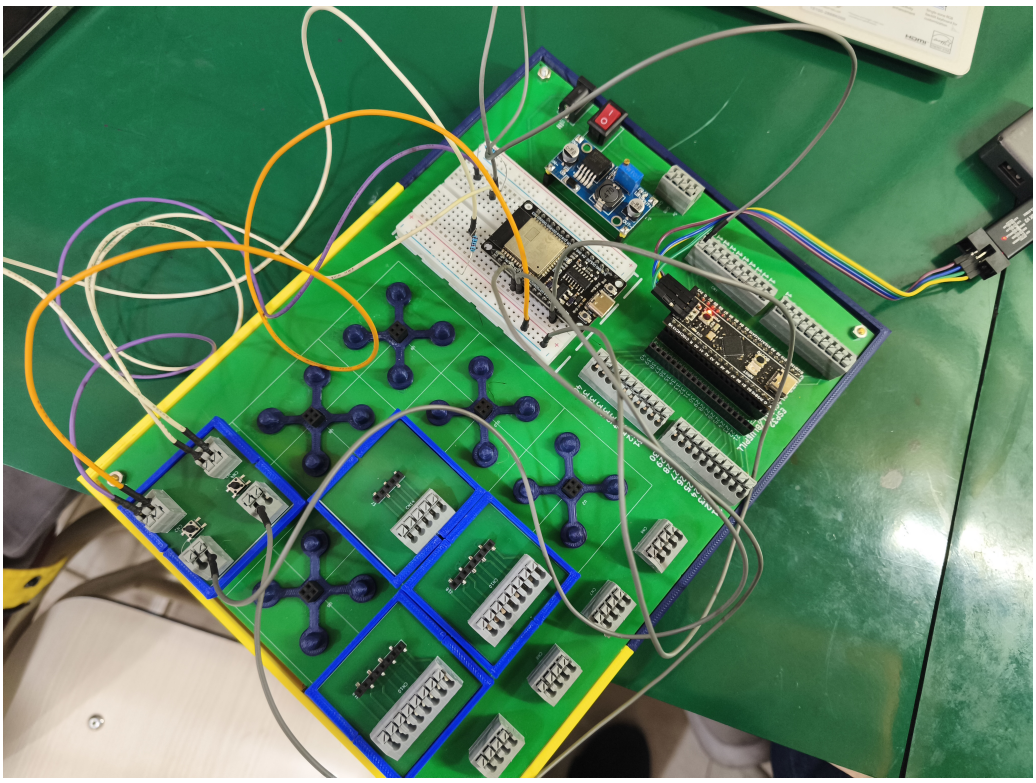


1.3 PERCOBAAN 3: Pull-up dan Pull-down: Eksternal vs Internal (ESP32 + Framework Arduino)

1. Ganti isi platformio.ini kembali ke konfigurasi Arduino di atas. Hapus src/main.c, lalu buat lagi file src/main.cpp



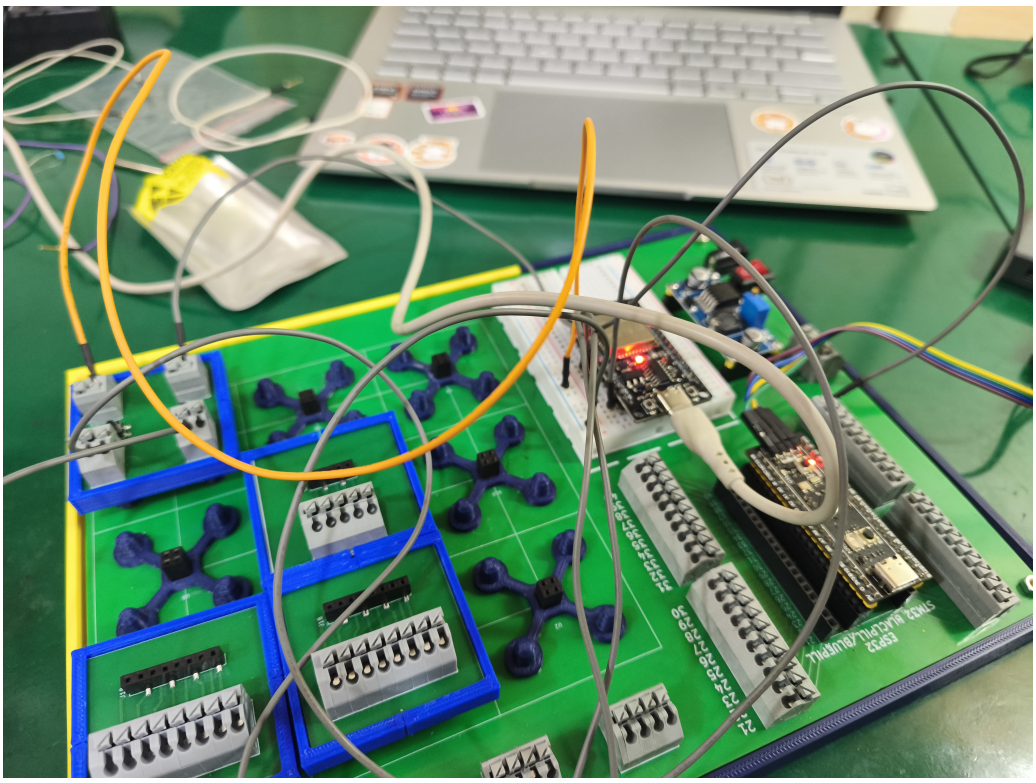
2. Pasang dua tombol di GPIO 32 dan 33 lengkap dengan resistornya, sesuai Skema Bagian A



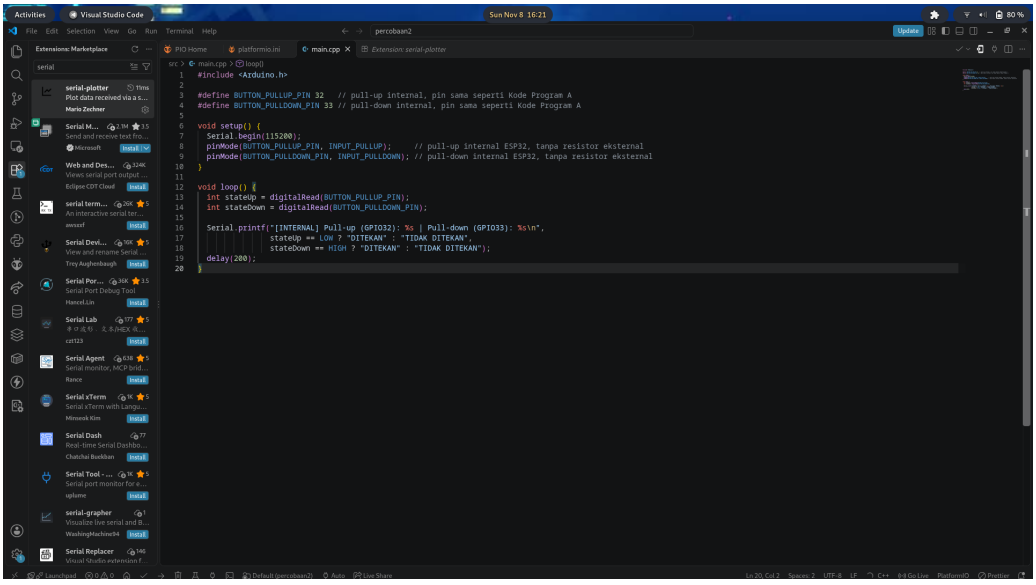
3. Tulis Kode Program A (Eksternal) di bawah pada src/main.cpp, klik Build dan Upload, lalu coba tekan kedua tombol dan catat hasilnya dari serial monitor.

```cpp
#include <Arduino.h>

#define BUTTON_PULLUP_PIN 32   // pull-up eksternal
#define BUTTON_PULLDOWN_PIN 33 // pull-down eksternal

void setup() {
  Serial.begin(115200);
  pinMode(BUTTON_PULLUP_PIN, INPUT);   // resistor pull-up dipasang eksternal, bukan INPUT_PULLUP internal
  pinMode(BUTTON_PULLDOWN_PIN, INPUT); // resistor pull-down dipasang eksternal
}

void loop() {
  int stateUp = digitalRead
  int stateDown = digitalRe
                                  IN);

  Serial.printf("[EKSTERNAL
                                  %s | Pull-down (GPIO33): %s\n",
                stateUp == LOW ? "DITEKAN" : "TIDAK DITEKAN",
                stateDown == HIGH ? "DITEKAN" : "TIDAK DITEKAN");
  delay(200);
}
```

4. Lepas kedua resistor sesuai Skema Bagian B (Gambar 7), tombol tetap terpasang. Lihat Gambar 8 dan Gambar 9 untuk memastikan pasangan kaki tombol yang benar

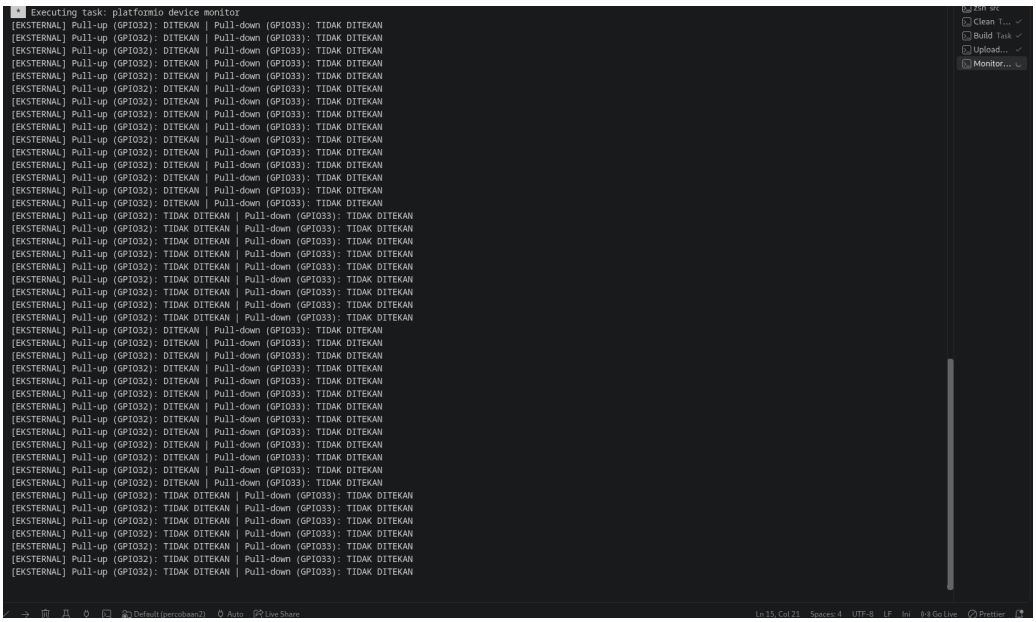


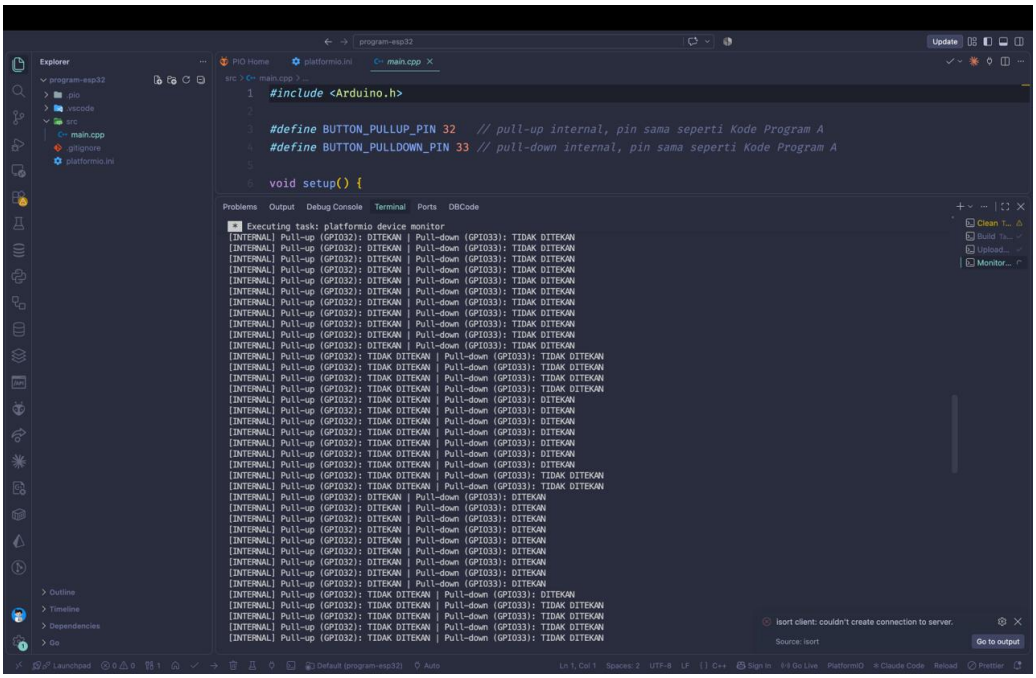
5. Ganti isi src/main.cpp dengan Kode Program B (Internal) di bawah, klik Build dan Upload, lalu coba tombol yang sama dan catat hasilnya dari serial monitor.



```
#include <Arduino.h>

#define BUTTON_PULLUP_PIN 32   // pull-up internal, pin sama seperti Kode Program A
#define BUTTON_PULLDOWN_PIN 33 // pull-down internal, pin sama seperti Kode Program A

void setup() {
  Serial.begin(115200);
  pinMode(BUTTON_PULLUP_PIN, INPUT_PULLUP);     // pull-up internal ESP32, tanpa resistor eksternal
  pinMode(BUTTON_PULLDOWN_PIN, INPUT_PULLDOWN); // pull-down internal ESP32, tanpa resistor eksternal
}

void loop() {
  int stateUp = digitalRead(BUTTON_PULLUP_PIN);
  int stateDown = digitalRead(BUTTON_PULLDOWN_PIN);

  Serial.printf("[INTERNAL] Pull-up (GPIO32): %s | Pull-down (GPIO33): %s\n",
                stateUp == LOW ? "DITEKAN" : "TIDAK DITEKAN",
                stateDown == HIGH ? "DITEKAN" : "TIDAK DITEKAN");
  delay(200);
}
```

6. Bandingkan hasil A dan B. Seharusnya sama: pull-up bernilai HIGH saat tidak ditekan dan LOW saat ditekan, pull-down kebalikannya

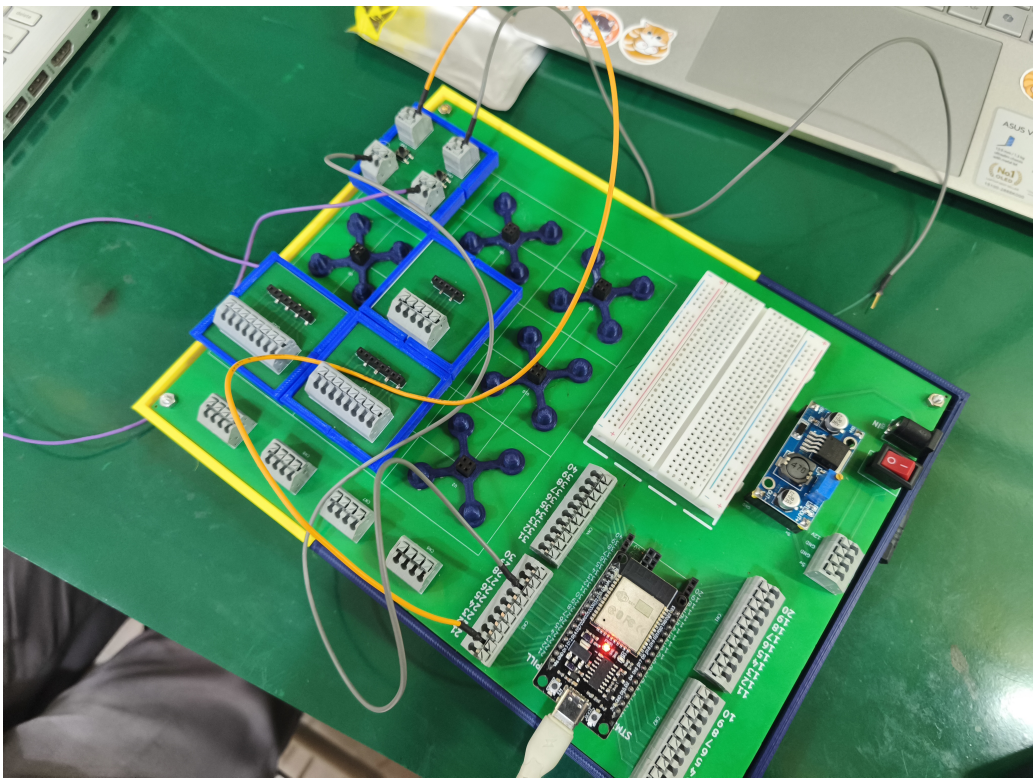




7. Jika hasilnya terbalik atau berbeda, periksa apakah kedua resistor sudah benar-benar dilepas

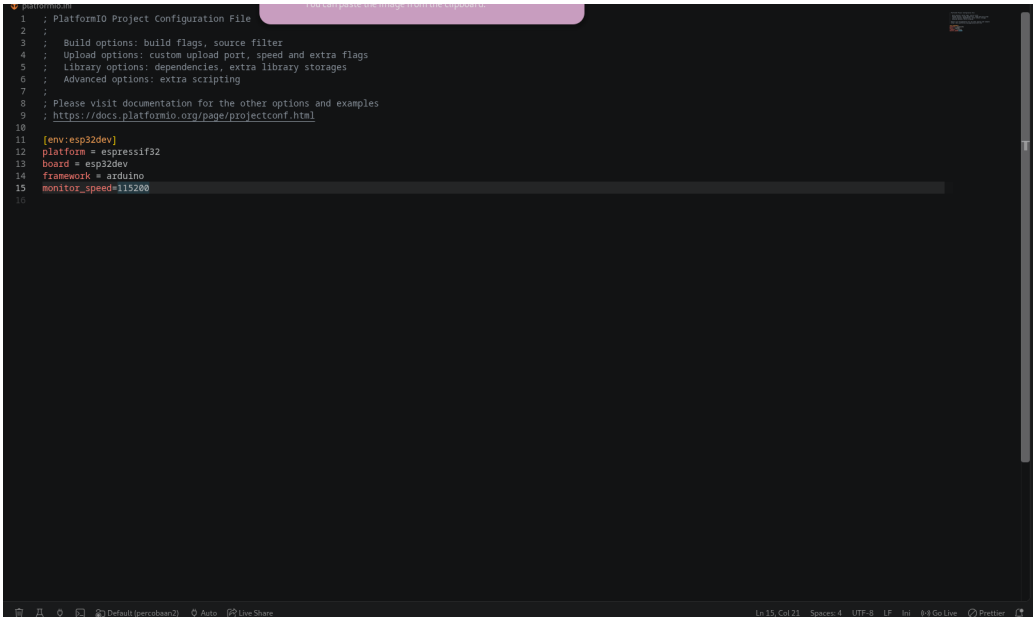
1.4 PERCOBAAN 4: Debouncing pada Input Tombol GPIO (ESP32 + Framework Arduino)

1. Pasang tombol sesuai skema di atas

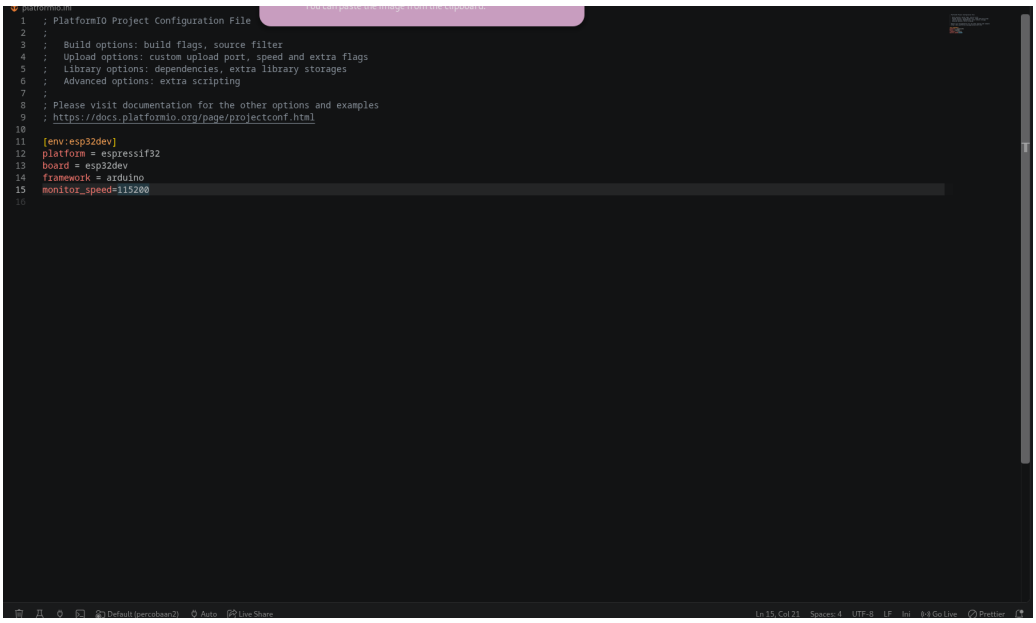


2. Tulis kode di bawah. Kode ini punya dua penghitung: counterNoDebounce (tanpa penyaring getaran tombol) dan counterWithDebounce (dengan penyaring)

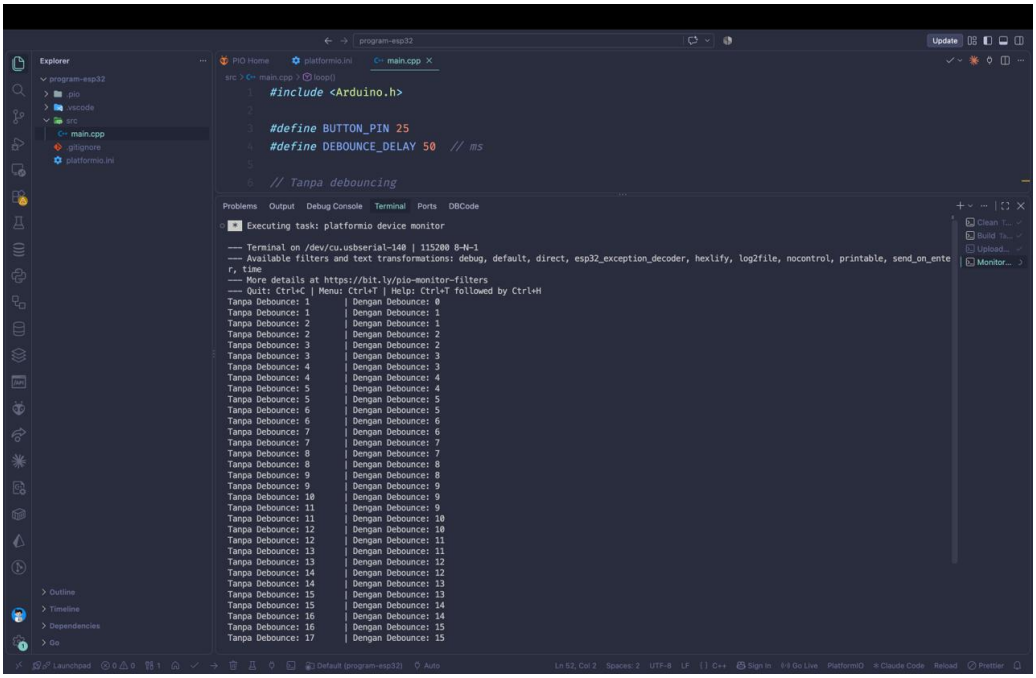
3. Tekan tombol satu kali, lalu bandingkan nilai kedua penghitung di Serial Monitor



4. Ulangi dengan cara menekan yang berbeda (cepat dan lambat). counterNoDebounce bisa naik lebih dari 1 untuk satu tekanan, sedangkan counterWithDebounce seharusnya naik tepat 1. Jika keduanya terlihat sama, coba tekan lagi karena getaran tombol tidak selalu muncul



5. Ubah DEBOUNCE_DELAY menjadi 10 ms, klik Build dan Upload, lalu tekan tombol lagi

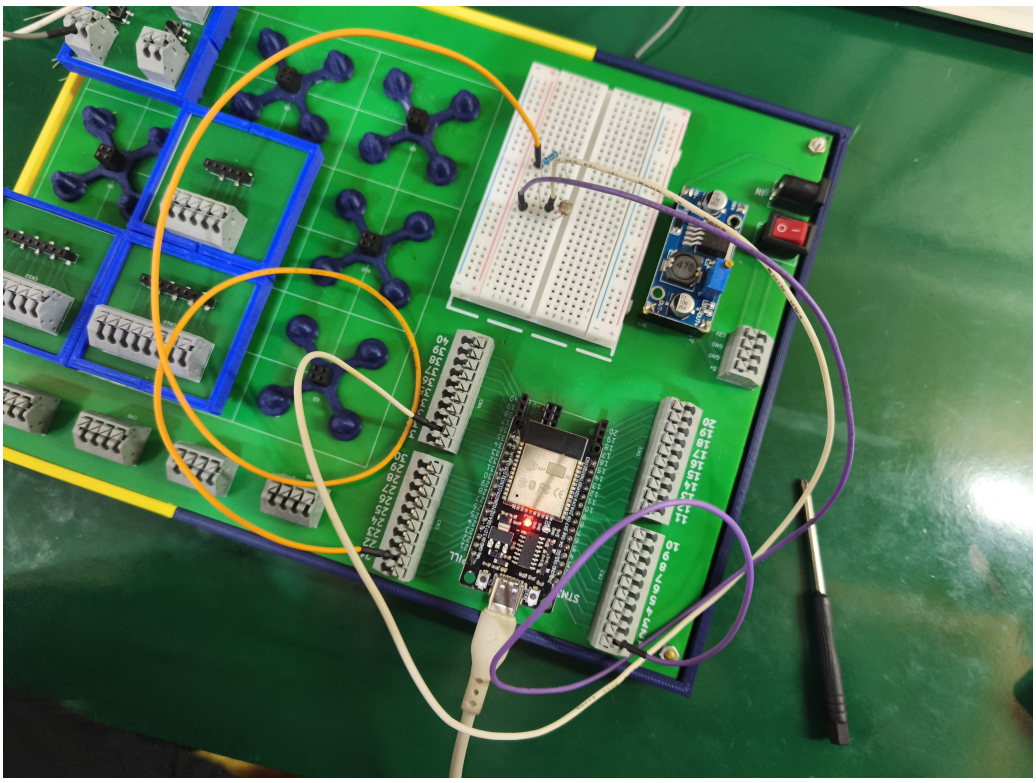


6. Ubah menjadi 200 ms klik Build dan Upload, lalu tekan tombol dengan cepat berulang kali

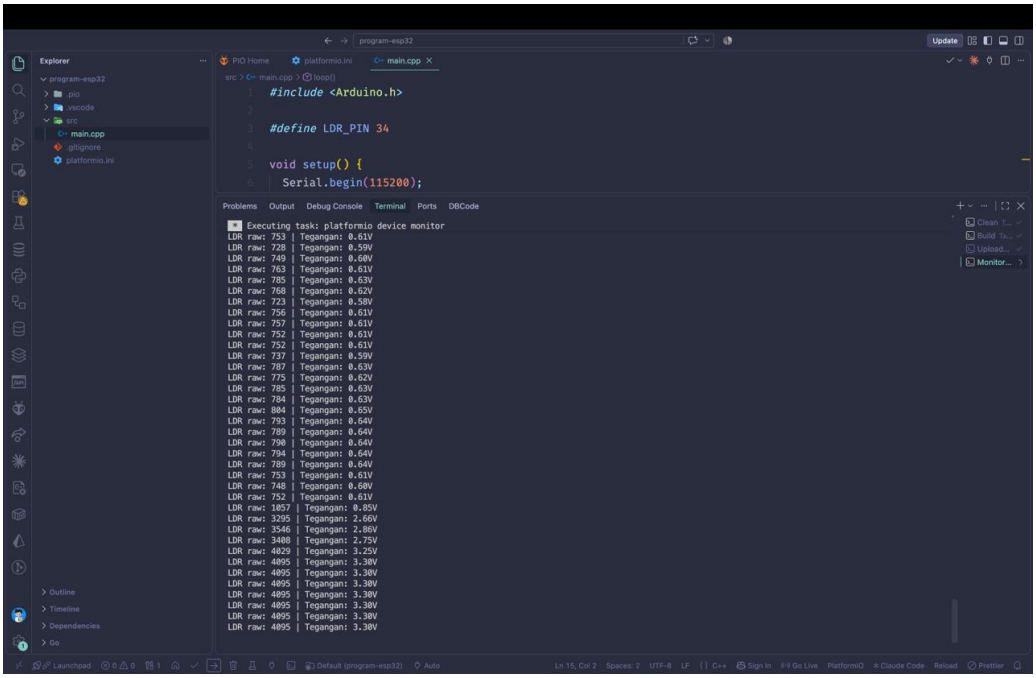
7. Bandingkan ketiga nilai 10,50,dan 200ms Analisis untung ruginya: makin besar nilainya makin bersih dari getaran, tetapi tombol makin lambat merespons

1.5 PERCOBAAN 5: Akses GPIO Analog: Pembacaan ADC dengan LDR (ESP32 + Framework Arduino)

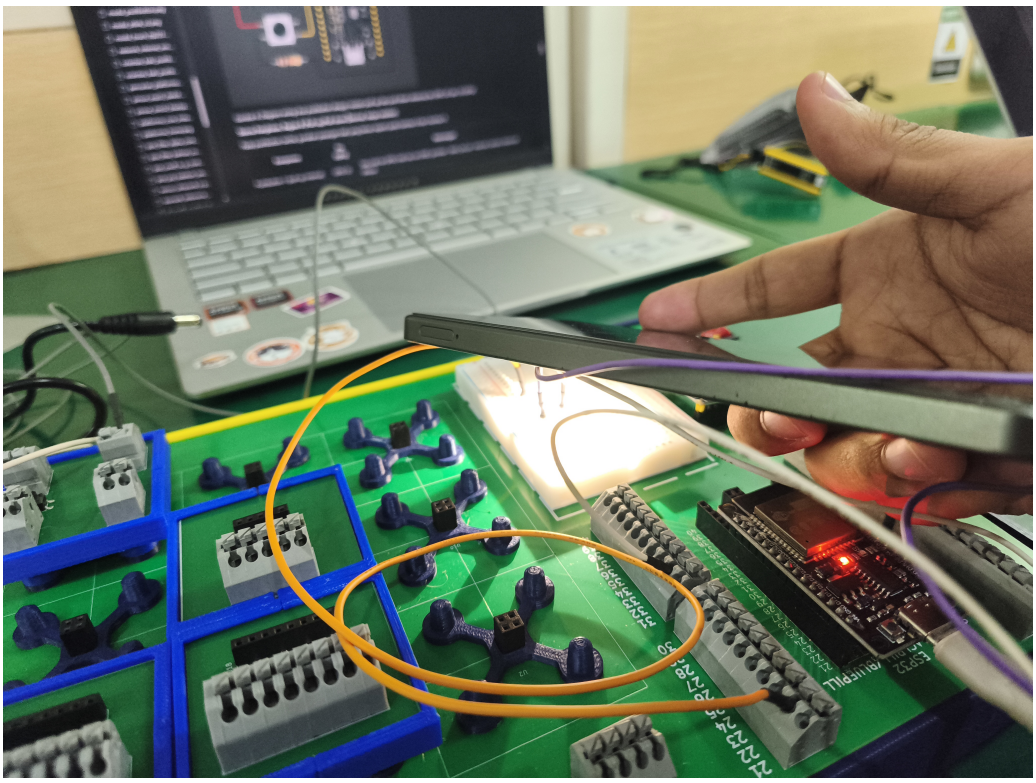
1. Pasang LDR bersama resistor sesuai skema, lalu hubungkan titik tengah keduanya ke GPIO 34



2. Tulis program yang membaca nilai ADC mentah dan mengubahnya menjadi tegangan
3. Klik Build dan Upload, lalu buka Serial Monitor



4. Tutup LDR dengan tangan, lalu sinari dengan lampu. Lihat nilai mentahnya (0 sampai 4095) berubah



5. Analisis mengapa sinyal analog bisa menunjukkan banyak kondisi, tidak hanya HIGH atau LOW

2 Analisis Hasil Percobaan

3 Analisis Hasil Percobaan

3.1 PERCOBAAN 1: Pengenalan STM32 Blackpill (Blink)

Pada percobaan pertama, proses pemrograman mikrokontroler STM32F411CEU6 (Blackpill) menggunakan antarmuka ST-Link V2 dan ekstensi PlatformIO berhasil dilakukan dengan baik. Setelah kode berhasil dikompilasi (*Build*) dan diunggah (*Upload*), LED internal yang terhubung pada pin `PC13` berkedip secara teratur dengan interval menyala selama 500 ms dan mati selama 500 ms secara terus-menerus.

Hal ini membuktikan bahwa konfigurasi `platformio.ini` untuk arsitektur ARM Cortex-M4 serta pengkabelan jalur komunikasi pemrograman (`SWDIO`, `SWCLK`, `GND`, dan $3{,}3$V) antara ST-Link dan papan STM32 Blackpill telah terhubung dengan benar. Selain itu, percobaan ini menunjukkan karakteristik khusus LED internal pada papan Blackpill yang bersifat *active-low*, di mana logika `LOW` (0V) akan mengalirkan arus sehingga LED menyala, sedangkan logika `HIGH` ($3{,}3$V) akan mematikannya.

3.2 PERCOBAAN 2: Pengenalan ESP32 dengan Framework ESP-IDF (Blink)

Percobaan kedua menunjukkan bahwa mikrokontroler ESP32 dapat diprogram secara langsung menggunakan *framework* native bawaan Espressif, yaitu ESP-IDF, dengan bahasa pemrograman C (`src/main.c`). Sistem berhasil mengenali port serial USB, mengunggah program, dan mengeksekusi tugas pengedipan LED secara stabil.

Berdasarkan hasil pengujian, terdapat beberapa perbedaan mendasar antara memprogram ESP32 menggunakan **ESP-IDF** dibandingkan dengan **framework Arduino**:

- **Struktur Program dan Sistem Operasi:** Arduino menyembunyikan kompleksitas sistem melalui fungsi abstraksi `setup()` dan `loop()`. Sebaliknya, ESP-IDF berjalan langsung di atas sistem operasi waktu nyata (*FreeRTOS*) dengan titik masuk utama `app_main()`, sehingga fungsi tunda tidak menggunakan `delay()`, melainkan fungsi manajemen tugas RTOS seperti `vTaskDelay()`.
- **Kontrol Perangkat Keras:** ESP-IDF memberikan akses kontrol perangkat keras yang lebih mendalam dan efisien (seperti konfigurasi arah pin melalui `gpio_set_direction()` dan pengelolaan memori maupun *multitasking* multi-core), sementara Arduino mengutamakan kemudahan sintaks melalui abstraksi tinggi (seperti `pinMode()` dan `digitalWrite()`).
- **Waktu Kompilasi:** Proses kompilasi awal pada ESP-IDF cenderung memakan waktu lebih lama dibandingkan Arduino karena harus membangun seluruh komponen inti RTOS dan pustaka sistem secara penuh.

3.3 PERCOBAAN 3: Pull-up dan Pull-down: Eksternal vs Internal (ESP32 + Framework Arduino)

Pada percobaan ketiga, pembacaan logika input digital dari dua buah *pushbutton* pada `GPIO 32` dan `GPIO 33` berhasil diamati melalui Serial Monitor. Pengujian menggunakan resistor eksternal (Kode Program A) maupun resistor internal mikrokontroler melalui mode `INPUT_PULLUP` dan `INPUT_PULLDOWN` (Kode Program B) memberikan hasil yang identik dan konsisten:

- Pada konfigurasi **Pull-Up**, kondisi bawaan (*idle*) saat tombol tidak ditekan terbaca sebagai logika `HIGH` (1) karena pin terhubung ke tegangan $3{,}3$V melalui resistor, dan berubah menjadi `LOW` (0) saat tombol ditekan karena terhubung langsung ke `GND`.
- Pada konfigurasi **Pull-Down**, kondisi bawaan saat tombol tidak ditekan terbaca sebagai logika `LOW` (0) karena pin ditarik ke `GND`, dan berubah menjadi `HIGH` (1) saat tombol ditekan karena terhubung ke jalur $3{,}3$V.

Keberhasilan ini membuktikan bahwa fitur resistor *pull-up* dan *pull-down* internal pada ESP32 berfungsi optimal untuk mencegah kondisi pin mengambang (*floating*) yang rentan terhadap derau elektromagnetik (*noise*), sehingga komponen resistor eksternal dapat dihemat dalam perancangan rangkaian.

3.4 PERCOBAAN 4: Debouncing pada Input Tombol GPIO (ESP32 + Framework Arduino)

Pada percobaan keempat, perbandingan antara pembacaan sinyal mentah (`counterNoDebounce`) dan pembacaan terfilter (`counterWithDebounce`) menunjukkan efektivitas algoritma *software debouncing*. Ketika tombol ditekan dengan kecepatan yang bervariasi, nilai `counterNoDebounce` teramati melonjak lebih dari satu hitungan dalam satu kali penekanan fisik akibat pantulan mekanis (*contact bounce*) pelat logam di dalam tombol. Sebaliknya, nilai `counterWithDebounce` selalu bertambah tepat satu angka setiap kali tombol ditekan.

Pengujian variasi nilai `DEBOUNCE_DELAY` menghasilkan analisis perbandingan performa sebagai berikut:

- **Delay** 10 **ms:** Respons tombol terasa sangat cepat dan instan, namun pada beberapa penekanan yang lambat atau tidak sempurna, pantulan mekanis yang berlangsung lebih dari 10 ms masih berpotensi lolos dan terbaca sebagai input ganda.
- **Delay** 50 **ms:** Memberikan keseimbangan paling optimal. Seluruh derau pantulan mekanis berhasil disaring sepenuhnya tanpa mengurangi responsivitas tombol saat ditekan secara normal oleh pengguna.
- **Delay** 200 **ms:** Sangat kebal terhadap getaran maupun pantulan mekanis, namun mengorbankan responsivitas sistem. Ketika tombol ditekan berulang kali dengan cepat, sebagian penekanan terabaikan karena masih berada di dalam jendela waktu tunda 200 ms.

3.5 PERCOBAAN 5: Akses GPIO Analog: Pembacaan ADC dengan LDR (ESP32 + Framework Arduino)

Pada percobaan kelima, rangkaian pembagi tegangan menggunakan sensor cahaya LDR (*Light Dependent Resistor*) dan resistor tetap yang dihubungkan ke pin analog `GPIO 34` berhasil dibaca oleh fitur ADC (*Analog-to-Digital Converter*) ESP32. Ketika sensor ditutup dengan tangan, nilai ADC mentah turun mendekati 0 (tegangan mendekati 0 V), dan ketika disinari cahaya lampu, nilai ADC meningkat hingga mendekati batas maksimal 4095 (tegangan mendekati $3{,}3$ V).

Sinyal analog mampu merepresentasikan beragam kondisi gradasi intensitas cahaya—tidak terbatas hanya pada dua keadaan biner (`HIGH` atau `LOW`)—karena cara kerja ADC yang melakukan proses kuantisasi tegangan kontinu. ESP32 memiliki modul ADC dengan resolusi **12-bit**, sehingga rentang tegangan input analog dari 0 V hingga $3{,}3$ V dipetakan ke dalam $2^{12} = 4096$ tingkat nilai diskrit (dari 0 hingga 4095), dengan ketelitian sekitar:

$$\text{Resolusi Tegangan} = \frac{3{,}3 \text{ V}}{4095} \approx 0{,}806 \text{ mV per langkah} \tag{1}$$

Karena perubahan intensitas cahaya menyebabkan nilai resistansi LDR berubah secara bertahap dan kontinu, tegangan keluaran pada titik tengah pembagi tegangan pun ikut bervariasi secara halus, yang kemudian diterjemahkan oleh ADC menjadi ribuan variasi nilai digital yang mewakili kondisi gelap, redup, normal, hingga sangat terang.

4 Hasil Tugas Modul

4.1 Matrix Pushbutton 3x3 Mengendalikan Matrix LED 3x3:

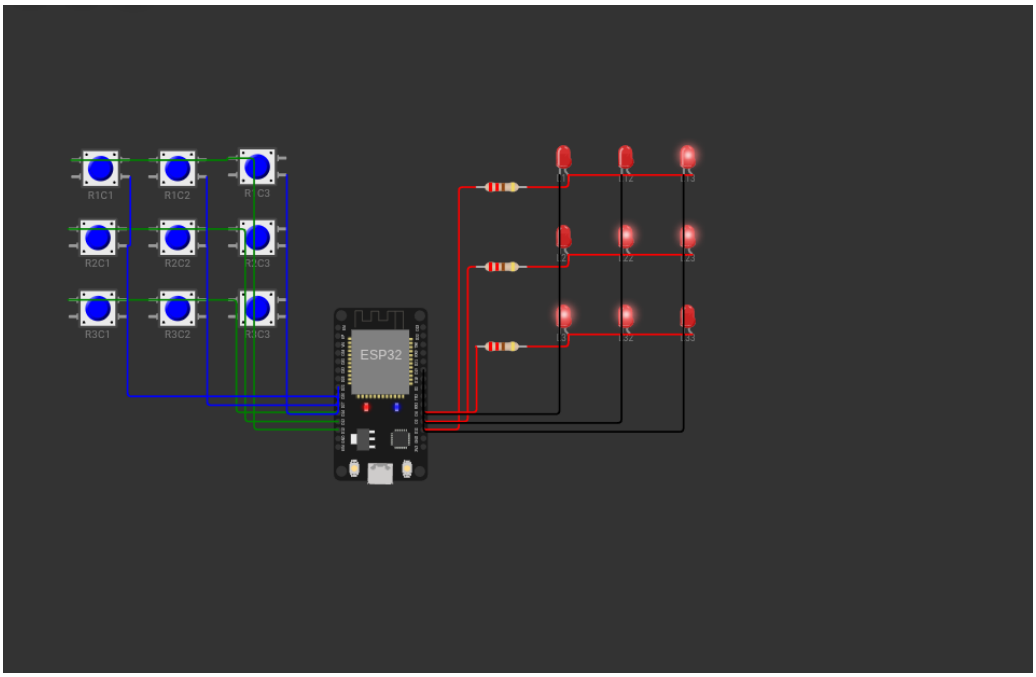
Buatlah simulasi untuk Matrix Pushbutton 3x3 yang mengendalikan Matrix LED 3x3, dimana tombol pada posisi (baris, kolom) tertentu menyalakan LED pada posisi yang sama, mis. tombol baris 1 kolom 1 menyalakan LED baris 1 kolom 1. Program wajib menerapkan debouncing pada setiap tombol (mengacu pada Percobaan 4), sehingga LED hanya berubah status setelah pembacaan tombol benar-benar stabil, bukan langsung mengikuti setiap perubahan sinyal mentah yang masih mungkin bouncing. Kumpulkan tangkapan layar dari hasil simulasi beserta source code-nya. (Simulasi dapat dilakukan menggunakan wokwi, cirkit, thinkercad, atau aplikasi simulasi lain yang mendukung mikrokontroler ESP32). Pertanyaan Analisis:

1. Analisis mengapa debouncing tetap diperlukan pada rangkaian matrix pushbutton ini, walaupun pada satu waktu hanya satu tombol yang biasanya ditekan, kaitkan dengan penyebab bouncing pada C.5 Debouncing

2. Analisis konfigurasi pull-up atau pull-down (sesuai rancangan Anda) yang digunakan pada pembacaan kolom tombol, apa kondisi (HIGH/LOW) pin tersebut saat tombol tidak ditekan, dan mengapa kondisi tersebut tidak boleh dibiarkan floating?

3. Andaikan salah satu Tombol pada matrix ini digantikan dengan LDR yang dibaca melalui ADC (seperti Percobaan 5), analisis perbedaan mendasar antara membaca GPIO sebagai digital input (tombol, hanya HIGH/LOW) dan sebagai analog input (ADC), mengapa ADC dapat merepresentasikan lebih dari dua kondisi?

4.2 JAWABAN:




```
// Definisi Pin Matrix Pushbutton 3x3
const int btnRowPins[3] = {13, 12, 14}; // Output scanning baris
const int btnColPins[3] = {27, 26, 25}; // Input pembacaan kolom (INPUT_PULLUP)

// Definisi Pin Matrix LED 3x3
const int ledRowPins[3] = {15, 2, 4};   // Anoda LED (+) -> Aktif HIGH
const int ledColPins[3] = {5, 18, 19};  // Katoda LED (-) -> Aktif LOW

// Variabel Debouncing untuk setiap tombol [baris][kolom]
bool buttonState[3][3];             // Status tombol yang sudah stabil
bool lastButtonReading[3][3];       // Pembacaan sinyal mentah sebelumnya
unsigned long lastDebounceTime[3][3]; // Waktu terakhir sinyal berubah
const unsigned long debounceDelay = 50; // Waktu tunda stabilisasi (50 ms)
```

```
// Status LED [baris][kolom] (true = NYALA, false = MATI)
bool ledState[3][3] = {
  {false, false, false},
  {false, false, false},
  {false, false, false}
};

void setup() {
  Serial.begin(115200);

  // Inisialisasi pin Matrix Pushbutton
  for (int r = 0; r < 3; r++) {
    pinMode(btnRowPins[r], OUTPUT);
    digitalWrite(btnRowPins[r], HIGH); // Default HIGH (non-aktif)
  }
  for (int c = 0; c < 3; c++) {
    pinMode(btnColPins[c], INPUT_PULLUP); // Aktif LOW saat tombol ditekan
  }

  // Inisialisasi status awal tombol
  for (int r = 0; r < 3; r++) {
    for (int c = 0; c < 3; c++) {
      buttonState[r][c] = HIGH;
      lastButtonReading[r][c] = HIGH;
      lastDebounceTime[r][c] = 0;
    }
  }

  // Inisialisasi pin Matrix LED
  for (int r = 0; r < 3; r++) {
    pinMode(ledRowPins[r], OUTPUT);
    digitalWrite(ledRowPins[r], LOW); // Anoda mati
  }
  for (int c = 0; c < 3; c++) {
    pinMode(ledColPins[c], OUTPUT);
    digitalWrite(ledColPins[c], HIGH); // Katoda mati (HIGH menahan arus)
  }

  Serial.println("Sistem Matrix 3x3 Siap. Tekan tombol untuk toggle LED.");
}

void loop() {
  scanButtonsWithDebounce();
  refreshLedMatrix();
```

```
}

// Fungsi 1: Pemindaian Matrix Tombol + Software Debouncing
void scanButtonsWithDebounce() {
  for (int r = 0; r < 3; r++) {
    // Aktifkan baris ke-r dengan menariknya ke LOW
    digitalWrite(btnRowPins[r], LOW);

    for (int c = 0; c < 3; c++) {
      // Baca kondisi mentah kolom ke-c
      bool reading = digitalRead(btnColPins[c]);

      // Jika terjadi perubahan sinyal (baik karena penekanan maupun bouncing/noise)
      if (reading != lastButtonReading[r][c]) {
        lastDebounceTime[r][c] = millis(); // Reset timer debounce
      }

      // Cek apakah sinyal sudah bertahan stabil melebihi debounceDelay (50 ms)
      if ((millis() - lastDebounceTime[r][c]) > debounceDelay) {
        // Jika status stabil berbeda dengan status tombol sebelumnya
        if (reading != buttonState[r][c]) {
          buttonState[r][c] = reading;

          // Aksi hanya dipicu saat transisi stabil menjadi LOW (tombol ditekan)
          if (buttonState[r][c] == LOW) {
            ledState[r][c] = !ledState[r][c]; // Toggle status LED yang bersesuaian
            Serial.printf("Tombol [Baris %d, Kolom %d] ditekan -> LED %s\n",
                          r + 1, c + 1, ledState[r][c] ? "ON" : "OFF");
          }
        }
      }

      // Simpan pembacaan mentah untuk siklus berikutnya
      lastButtonReading[r][c] = reading;
    }

    // Kembalikan baris ke-r menjadi HIGH sebelum pindah ke baris berikutnya
    digitalWrite(btnRowPins[r], HIGH);
  }
}

// Fungsi 2: Multiplexing Matrix LED 3x3
void refreshLedMatrix() {
  for (int r = 0; r < 3; r++) {
```

```
    // Atur semua kolom LED sesuai data ledState pada baris ke-r
    for (int c = 0; c < 3; c++) {
      if (ledState[r][c]) {
        digitalWrite(ledColPins[c], LOW);  // Aktifkan katoda (-) untuk menyalakan
      } else {
        digitalWrite(ledColPins[c], HIGH); // Nonaktifkan katoda (-)
      }
    }

    // Nyalakan anoda (+) baris ke-r sesaat
    digitalWrite(ledRowPins[r], HIGH);
    delayMicroseconds(1500); // Tahan 1.5 ms agar mata manusia melihat cahaya kontinu (POV)
    digitalWrite(ledRowPins[r], LOW);  // Matikan baris sebelum pindah (mencegah ghosting)
  }
}
```

1. Meskipun hanya satu tombol yang ditekan, sumber masalah bouncing bukan berasal dari jumlah tombol yang ditekan bersamaan, melainkan dari sifat mekanis kontak logam di dalam masing-masing tombol (tactile switch).

    - Penyebab Mekanis Bouncing: Ketika tombol ditekan atau dilepas, pelat pegas logam di dalamnya tidak langsung menempel atau terpisah dengan sempurna dalam satu gerakan mulus. Pelat tersebut mengalami pantulan mikro (contact bounce) selama beberapa milidetik (umumnya 5–20 ms), yang menyebabkan rangkaian terhubung dan terputus berkali-kali secara sangat cepat.
    - Dampak pada Sistem Matrix: Mikrokontroler ESP32 melakukan pemindaian (scanning) baris dan kolom dengan kecepatan sangat tinggi (ribuan kali per detik). Tanpa debouncing, satu kali tekanan fisik dari jari pengguna akan terbaca oleh ESP32 sebagai rentetan pulsa HIGH-LOW-HIGH-LOW berulang kali. Pada program kontrol LED (terutama mode toggle atau pencacah), pantulan ini akan membuat LED berkedip tidak menentu atau berakhir pada status yang salah setelah jari dilepas.

2. Meskipun hanya satu tombol yang ditekan, sumber masalah bouncing bukan berasal dari jumlah tombol yang ditekan bersamaan, melainkan dari sifat mekanis kontak logam di dalam masing-masing tombol (tactile switch).

    - Konfigurasi yang Digunakan: Rancangan ini menggunakan konfigurasi Internal Pull-Up pada pin kolom GPIO 27, 26, 25, sedangkan pin baris bertindak sebagai pemicu aktif rendah.
    - Kondisi Saat Tombol Tidak Ditekan: Saat tombol tidak ditekan (rangkaian terbuka), resistor pull-up internal menghubungkan pin kolom ke sumber tegangan logika tinggi (3.3V), sehingga pin terbaca dalam kondisi HIGH (logika 1). Ketika baris ditarik ke LOW (0V) dan tombol pada persimpangan tersebut ditekan, pin kolom terhubung langsung ke pin baris tersebut sehingga tegangannya turun menjadi LOW.
    - Mengapa Tidak Boleh Dibiarkan Floating: Jika pin input dikonfigurasi tanpa pull-up maupun pull-down (INPUT biasa), saat tombol tidak ditekan pin tersebut berada dalam keadaan

mengambang (floating) atau impedansi tinggi tanpa referensi tegangan yang pasti. Akibatnya, pin akan sangat sensitif terhadap derau elektromagnetik (noise), listrik statis di sekitar rangkaian, maupun kapasitansi parasitik antar kabel jumper, sehingga pembacaan logika akan berubah-ubah secara acak antara HIGH dan LOW meskipun tidak ada tombol yang disentuh (false triggering).

- Perbedaan Mendasar (Digital Input vs. Analog Input/ADC):
  - Digital Input (Tombol): Bekerja menggunakan rangkaian komparator ambang batas tegangan (logic threshold). Mikrokontroler hanya mengklasifikasikan tegangan masuk ke dalam dua keadaan diskrit: di bawah ambang batas tertentu dianggap LOW (0) (misalnya < 0.8V pada ESP32) dan di atas ambang batas dianggap HIGH (1) (misalnya > 2.0V). Nilai tegangan di antara rentang tersebut tidak diukur besarannya secara spesifik.
  - Analog Input / ADC (Analog-to-Digital Converter): Mengukur besar tegangan listrik kontinu secara presisi (pada ESP32 rentangnya dari 0V hingga 3.3V) dan mengonversinya menjadi bilangan bulat digital (kuantisasi).
- Mengapa ADC Dapat Merepresentasikan Lebih dari Dua Kondisi: ADC memiliki resolusi bit tertentu untuk membagi rentang tegangan referensi menjadi banyak tingkatan diskrit. Pada ESP32, ADC memiliki resolusi standar 12-bit, yang berarti rentang tegangan 0 V $-$ $3{,}3$ V dibagi menjadi $2^{12} = 4096$ tingkat nilai digital (dari 0 hingga 4095), dengan ketelitian sekitar $\frac{3{,}3\text{ V}}{4095} \approx 0{,}8$ mV per langkah. Karena LDR menghasilkan perubahan resistansi (dan pembagi tegangan) secara bertahap sesuai intensitas cahaya, ADC mampu membedakan gradasi kondisi dari sangat gelap (nilai mendekati 0), redup, cukup terang, hingga sangat terang (mendekati 4095), bukan sekadar kondisi “gelap” atau “terang” saja.

5 Kesimpulan

Berdasarkan rangkaian percobaan yang telah dilaksanakan pada Modul 1, dapat disimpulkan bahwa pemrograman dasar antarmuka perangkat keras maupun pengelolaan masukan dan keluaran pada mikrokontroler STM32 Blackpill serta ESP32 telah berhasil dijalankan dengan baik menggunakan PlatformIO. Pemilihan *framework* pengembangan memengaruhi tingkat kendali sistem, di mana ESP-IDF memberikan akses langsung terhadap perangkat keras berbasis sistem operasi waktu nyata (*FreeRTOS*), sedangkan *framework* Arduino menawarkan abstraksi sintaks yang lebih ringkas untuk mempercepat proses implementasi. Pada pengelolaan input digital, penggunaan resistor *pull-up* dan *pull-down*—baik secara eksternal maupun melalui fitur internal mikrokontroler (`INPUT_PULLUP` dan `INPUT_PULLDOWN`)—terbukti krusial untuk menjaga stabilitas logika referensi dan mencegah kondisi pin mengambang (*floating*) akibat derau elektromagnetik. Selain itu, penerapan algoritma *software debouncing* dengan jeda stabilisasi optimal sebesar 50 ms mampu menghilangkan efek pantulan mekanis (*contact bounce*) pada tombol tanpa mengorbankan kecepatan respons sistem. Terakhir, berbeda dengan pembacaan digital yang hanya mengenali dua keadaan logika (`HIGH` dan `LOW`), penggunaan fitur *Analog-to-Digital Converter* (ADC) beresolusi 12-bit pada ESP32 mampu mengkuantisasi perubahan tegangan kontinu dari sensor cahaya LDR ke dalam 4096 tingkat nilai diskrit (0 hingga 4095), sehingga memungkinkan sistem mendeteksi berbagai gradasi kondisi lingkungan secara lebih presisi.

6 Lampiran

6.1 Dokumentasi saat praktikum